

人工智能基础实验四

实验报告

实验题目: 朴素贝叶斯算法的实现与应用

专 业: 人工智能

姓 名: 马承乾

学 号: 22920212204180

实验日期: 2023.12.7

目录

- 一. 实验题目3
- 二. 实验目的3
- 三. 实验方法3
 - 朴素贝叶斯原理3
 - 全流程概述4
 - 详细步骤介绍4
- 四. 实验结果7
- 五. 实验分析7
- 六. 附录8

一. 实验题目

朴素贝叶斯算法实现与性能分析

二. 实验目的

- 理解朴素贝叶斯算法的基本原理和实现过程。
- 学习如何使用 Python 实现朴素贝叶斯算法。
- 通过实验，掌握朴素贝叶斯算法在文本分类任务中的应用。
- 分析和比较不同参数对朴素贝叶斯算法性能的影响。

这个实验的主要目标是深入了解朴素贝叶斯算法，并通过实际编码和实验分析来加深对算法原理的理解。实验中将通过一个简单的文本分类任务来应用该算法，并通过性能分析来评估算法的效果。同时，通过调节参数，展示实验结果，更直观地了解算法在不同情境下的表现。

三. 实验方法

朴素贝叶斯原理

朴素贝叶斯 (Naive Bayes) 算法是一类基于贝叶斯定理的分类算法。其基本思想是通过训练数据学习类别之间的联合概率分布，然后利用贝叶斯定理进行分类。具体而言，朴素贝叶斯算法假设特征之间是相互独立的，即一个特征的出现不受其他特征的影响。这是一个朴素的假设，因此称为"朴素"贝叶斯。

1 贝叶斯定理：

贝叶斯定理描述了在已知先验概率的情况下，如何通过新的观察数据来更新我们对事件概率的估计。对于分类问题，我们希望计算给定特征向量的情况下，每个类别的后验概率，然后选择具有最高后验概率的类别。

贝叶斯定理的公式为： $P(A|B) = P(B|A)P(A)/P(B)$

其中， $P(A|B)$ 是在给定观察数据 B 的情况下事件 A 的后验概率， $P(B|A)$ 是在事件 A 的情况下观察到数据 B 的概率， $P(A)$ 是事件 A 的先验概率， $P(B)$ 是观察到数据 B 的概率。

2 朴素假设：

朴素贝叶斯算法对特征之间的独立性做了一个朴素的假设，即在给定类别的情况下，特征之间是相互独立的。这意味着可以将联合概率分布简化为各个特征的条件概率的乘积。

对于类别 c 和特征向量 x ，分类问题的基本公式为：

$$P(c|x) \propto P(c) \prod_{i=1}^n P(x_i|c)$$

其中， $P(c)$ 是类别 c 的先验概率， $P(x_i|c)$ 是在类别 c 的情况下特征 x_i 的条件概率。

3 分类决策规则：

朴素贝叶斯分类器使用最大后验概率准则进行分类决策。对于给定的特征向量 xx ，选择具有最大后验概率 $P(c|x)$ 的类别作为最终的分类结果：

$$\hat{y} = \arg \max_c P(c|x)$$

朴素贝叶斯算法通常用于文本分类、垃圾邮件过滤等领域，它的简单性和高效性使得它成为许多实际应用中的一种常用分类方法。

🚦 全流程概述

- ✧ 准备数据： 构建实验数据集，包括文档和对应的类别标签。
- ✧ 数据预处理： 将文档数据转换为朴素贝叶斯算法所需的格式，生成词汇表和训练集向量。
- ✧ 训练模型： 使用训练集数据训练朴素贝叶斯分类器，得到条件概率和先验概率。
- ✧ 测试模型： 使用测试集数据对训练好的模型进行测试，评估模型性能。
- ✧ 性能分析： 分析实验结果，观察不同参数对算法性能的影响。

🚦 详细步骤介绍

1. 准备数据： 构建实验数据集，包括文档和对应的类别标签。
构建一个包含文档和类别标签的数据集。每个文档是一个词条列表，类别标签表示文档属于侮辱类（1）还是非侮辱类（0）。
2. 数据预处理： 将文档数据转换为朴素贝叶斯算法所需的格式，生成词汇表和训练集向量。
创建词汇表： 对整个数据集进行遍历，将所有词条构建成一个词汇表，去除重复词条

$$\text{vocabSet} = \bigcup_{i=1}^N \text{set}(\text{doc}_i)$$

具体代码如下：

```
def create_vocab_list(data_set):  
    """  
    创建词汇表  
  
    Args:  
        data_set (list): 包含文档的列表  
  
    Returns:  
        list: 词汇表  
    """  
    vocab_set = set()  
    for doc in data_set:  
        vocab_set |= set(doc)  
    return list(vocab_set)
```

3. 生成训练集向量： 遍历每个文档，将文档转换为词条向量（0 和 1 构成的向量，表示词汇表中的词条是否在文档中出现）。

$$\text{returnVec}_i[j] = \begin{cases} 1, & \text{if word}_j \text{ in inputSet}_i \\ 0, & \text{otherwise} \end{cases}$$

具体代码如下：

```
def get_train_mat(data_set):
    """
    生成训练集向量列表

    Args:
        data_set (list): 包含文档的列表

    Returns:
        list: 训练集向量列表
    """
    train_mat = []
    vocab_list = create_vocab_list(data_set)
    for input_set in data_set:
        return_vec = set_of_words_to_vec(vocab_list, input_set)
        train_mat.append(return_vec)
    return train_mat
```

4. 训练模型：使用训练集数据训练伯努利朴素贝叶斯分类器，得到条件概率和先验概率。
计算先验概率： 计算文档属于侮辱类的概率 $P(\text{Abusive})$ 。

$$P(\text{Abusive}) = \frac{\sum_{i=1}^N \text{classVec}_i}{N}$$

计算条件概率： 对于每个词条，分别计算在侮辱类和非侮辱类中的条件概率。

$$P(\text{word}_j|\text{Abusive}) = \frac{\sum_{i=1}^N \text{word}_j \text{ in doc}_i \text{ and classVec}_i=1}{\sum_{i=1}^N \text{classVec}_i}$$

$$P(\text{word}_j|\text{Not Abusive}) = \frac{\sum_{i=1}^N \text{word}_j \text{ in doc}_i \text{ and classVec}_i=0}{\sum_{i=1}^N (1-\text{classVec}_i)}$$

计算对数概率： 由于概率相乘变为对数概率相加，防止下溢问题。

$$\log(P(\text{word}_j|\text{Abusive}))$$

$$\log(P(\text{word}_j|\text{Not Abusive}))$$

代码：

```
def train_naive_bayes(train_mat, class_labels):
    """
    朴素贝叶斯分类器训练函数

    Args:
        train_mat (list): 训练集向量列表
        class_labels (list): 类别标签向量

    Returns:
```

```

    tuple: 包含属于非侮辱类、侮辱类和文档属于侮辱类的概率的对数的元组
    """
    num_docs = len(train_mat)          # 计算训练的文档数目
    num_words = len(train_mat[0])      # 计算每篇文档的词条数
    p_abusive = sum(class_labels) / num_docs  # 文档属于侮辱类的概率
    p0_num, p1_num = np.ones(num_words), np.ones(num_words)
    p0_denom, p1_denom = 2, 2

    for i in range(num_docs):
        if class_labels[i] == 1:
            p1_num += train_mat[i]
            p1_denom += sum(train_mat[i])
        else:
            p0_num += train_mat[i]
            p0_denom += sum(train_mat[i])

    p1_v = np.log(p1_num / p1_denom)
    p0_v = np.log(p0_num / p0_denom)

    return p0_v, p1_v, p_abusive

```

5. 测试模型： 使用测试集数据对训练好的模型进行测试，评估模型性能。
 向量化测试样本： 将测试文档转换为词条向量。

$$\text{testVector}_i[j] = \begin{cases} 1, & \text{if word}_j \text{ in testVec}_i \\ 0, & \text{otherwise} \end{cases}$$

进行分类： 计算文档属于侮辱类和非侮辱类的概率，选择概率较大的类别。

$$\begin{aligned}
 P(\text{Abusive}|\text{testVector}) &= P(\text{Abusive}) \prod_{j=1}^M P(\text{word}_j|\text{Abusive}) \\
 P(\text{Not Abusive}|\text{testVector}) &= \\
 P(\text{Not Abusive}) \prod_{j=1}^M P(\text{word}_j|\text{Not Abusive})
 \end{aligned}$$

代码如下：

```

def classify_naive_bayes(vec_to_classify, p0_v, p1_v, p_abusive):
    """
    朴素贝叶斯分类函数

    Args:
        vec_to_classify (list): 待分类的文档向量
        p0_v (numpy.ndarray): 属于非侮辱类的条件概率的对数
        p1_v (numpy.ndarray): 属于侮辱类的条件概率的对数
        p_abusive (float): 文档属于侮辱类的概率

    Returns:
        int: 分类结果，1 表示属于侮辱类，0 表示属于非侮辱类
    """

```

```
"""
p1 = sum(vec_to_classify * p1_v) + np.log(p_abusive)
p0 = sum(vec_to_classify * p0_v) + np.log(1 - p_abusive)
# 用对数可以解决小数相乘带来的下溢问题
return 1 if p1 > p0 else 0
```

6. 输出分类结果： 根据分类概率，判断文档属于侮辱类还是非侮辱类。
7. 性能分析： 分析实验结果，观察不同参数对算法性能的影响。
此处可更改的参数为初始化平滑参数(用来防止分母为 0)，具体调参结果见实验结果。

四. 实验结果

在给定的测试用例下，伯努利朴素贝叶斯分类器的分类结果如下（具体用例见附录完整代码）：

- 测试用例 1（正面文档）：分类结果正确，属于非侮辱类。
- 测试用例 2（负面文档）：分类结果正确，属于侮辱类。
- 测试用例 3（包含所有正面词汇）：分类结果正确，属于非侮辱类。
- 测试用例 4（包含所有负面词汇）：分类结果正确，属于侮辱类。
- 测试用例 5（包含正面和负面词汇）：分类结果正确，属于非侮辱类。
- 测试用例 6（包含重复词汇）：分类结果正确，属于非侮辱类。

#	测试用例编号	实际类别	分类结果
#	1	非侮辱类	非侮辱类
#	2	侮辱类	侮辱类
#	3	非侮辱类	非侮辱类
#	4	侮辱类	侮辱类
#	5	非侮辱类	非侮辱类
#	6	非侮辱类	非侮辱类

通过调整平滑参数等参数，观察到对分类器性能的影响，我将初始化的平滑参数从都为 1 修改至都为 100，其间步长为 10，均为发现结果变化，可见该方法对参数敏感度不高。

五. 实验分析

在实验结果中，我发现该方法对平滑参数敏感度低，猜测原因，可能是朴素贝叶斯方法的假设（各个条件相互独立、且在文本处理中词汇出现的次数、顺序均不被考虑）已经是很强的假设，所以对其他参数的敏感度才比较低。

伯努利朴素贝叶斯分类器在文本分类任务中表现良好，对于简单的文本样本具有较高的分类准确性。通过实验，我们深入了解了该算法的原理和应用，为进一步研究和应用提供了基础。在实际应用中，可以根据具体任务和数据特点对算法参数进行调整，以达到更好的性能。

六. 附录

```
import numpy as np

def load_data_set():
    """
    创建实验数据集

    Returns:
        tuple: 包含实验数据集和类别标签向量的元组
    """
    data_set = [
        ['my', 'dog', 'has', 'flea', 'problems', 'help', 'please'],
        ['maybe', 'not', 'take', 'him', 'to', 'dog', 'park', 'stupid'],
        ['my', 'dalmation', 'is', 'so', 'cute', 'I', 'love', 'him'],
        ['stop', 'posting', 'stupid', 'worthless', 'garbage'],
        ['mr', 'licks', 'ate', 'my', 'steak', 'how', 'to', 'stop',
        'him'],
        ['quit', 'buying', 'worthless', 'dog', 'food', 'stupid']
    ]
    class_labels = [0, 1, 0, 1, 0, 1] # 类别标签向量, 1 代表侮辱性词汇, 0
    代表非侮辱性词汇
    return data_set, class_labels

def create_vocab_list(data_set):
    """
    创建词汇表

    Args:
        data_set (list): 包含文档的列表

    Returns:
        list: 词汇表
    """
    vocab_set = set()
    for doc in data_set:
        vocab_set |= set(doc)
    return list(vocab_set)

def set_of_words_to_vec(vocab_list, input_set):
    """
    将词条转换为词条向量
```

```

    Args:
        vocab_list (list): 词汇表
        input_set (list): 输入的词条列表

    Returns:
        list: 词条向量
    """
    return_vec = [0] * len(vocab_list)
    for word in input_set:
        if word in vocab_list:
            return_vec[vocab_list.index(word)] = 1
        else:
            print(f"{word} is not in my Vocabulary!")
    return return_vec

def get_train_mat(data_set):
    """
    生成训练集向量列表

    Args:
        data_set (list): 包含文档的列表

    Returns:
        list: 训练集向量列表
    """
    train_mat = []
    vocab_list = create_vocab_list(data_set)
    for input_set in data_set:
        return_vec = set_of_words_to_vec(vocab_list, input_set)
        train_mat.append(return_vec)
    return train_mat

def train_naive_bayes(train_mat, class_labels):
    """
    朴素贝叶斯分类器训练函数

    Args:
        train_mat (list): 训练集向量列表
        class_labels (list): 类别标签向量

    Returns:
        tuple: 包含属于非侮辱类、侮辱类和文档属于侮辱类的概率的对数的元组
    """
    num_docs = len(train_mat)                # 计算训练的文档数目

```

```

num_words = len(train_mat[0])      # 计算每篇文档的词条数
p_abusive = sum(class_labels) / num_docs  # 文档属于侮辱类的概率
p0_num, p1_num = np.ones(num_words), np.ones(num_words)
p0_denom, p1_denom = 2, 2

for i in range(num_docs):
    if class_labels[i] == 1:
        p1_num += train_mat[i]
        p1_denom += sum(train_mat[i])
    else:
        p0_num += train_mat[i]
        p0_denom += sum(train_mat[i])

p1_v = np.log(p1_num / p1_denom)
p0_v = np.log(p0_num / p0_denom)

return p0_v, p1_v, p_abusive

def classify_naive_bayes(vec_to_classify, p0_v, p1_v, p_abusive):
    """
    朴素贝叶斯分类函数

    Args:
        vec_to_classify (list): 待分类的文档向量
        p0_v (numpy.ndarray): 属于非侮辱类的条件概率的对数
        p1_v (numpy.ndarray): 属于侮辱类的条件概率的对数
        p_abusive (float): 文档属于侮辱类的概率

    Returns:
        int: 分类结果, 1 表示属于侮辱类, 0 表示属于非侮辱类
    """
    p1 = sum(vec_to_classify * p1_v) + np.log(p_abusive)
    p0 = sum(vec_to_classify * p0_v) + np.log(1 - p_abusive)
    # 用对数可以解决小数相乘带来的下溢问题
    return 1 if p1 > p0 else 0

def test_naive_bayes(test_vector):
    """
    朴素贝叶斯测试函数

    Args:
        test_vector (list): 待测试的文档向量
    """
    # 加载数据集和类别标签向量

```

```

data_set, class_labels = load_data_set()

# 创建词汇表
vocab_list = create_vocab_list(data_set)

# 生成训练集向量列表
training_matrix = get_train_mat(data_set)

# 训练朴素贝叶斯分类器
p0_vector, p1_vector, p_ab = train_naive_bayes(training_matrix,
class_labels)

# 将测试样本向量化
test_vectorized = set_of_words_to_vec(vocab_list, test_vector)

# 进行分类
classification_result = classify_naive_bayes(test_vectorized,
p0_vector, p1_vector, p_ab)

# 输出分类结果
category = "侮辱类" if classification_result == 1 else "非侮辱类"
print(f"{test_vector} 属于 {category}")

# 测试用例 1
test_vec1 = ['love', 'my', 'dalmation']
test_naive_bayes(test_vec1)

# 测试用例 2
test_vec2 = ['stupid', 'garbage']
test_naive_bayes(test_vec2)

# 测试用例 3: 包含所有正面词汇的例子
test_vec3 = ['my', 'dog', 'is', 'cute', 'I', 'love', 'him']
test_naive_bayes(test_vec3)

# 测试用例 4: 包含所有负面词汇的例子
test_vec4 = ['stupid', 'worthless', 'garbage', 'posting']
test_naive_bayes(test_vec4)

# 测试用例 5: 包含正面和负面词汇的例子
test_vec5 = ['my', 'dog', 'is', 'stupid', 'I', 'love', 'him']
test_naive_bayes(test_vec5)

# 测试用例 6: 包含重复词汇的例子

```

```
test_vec6 = ['my', 'dog', 'dog', 'problems', 'help', 'please']
test_naive_bayes(test_vec6)
```

```
# +-----+-----+-----+
# | 测试用例编号 | 实际类别 | 分类结果 |
# +-----+-----+-----+
# | 1           | 非侮辱类 | 非侮辱类 |
# | 2           | 侮辱类   | 侮辱类   |
# | 3           | 非侮辱类 | 非侮辱类 |
# | 4           | 侮辱类   | 侮辱类   |
# | 5           |          | 非侮辱类 |
# | 6           | 非侮辱类 | 非侮辱类 |
# +-----+-----+-----+
```